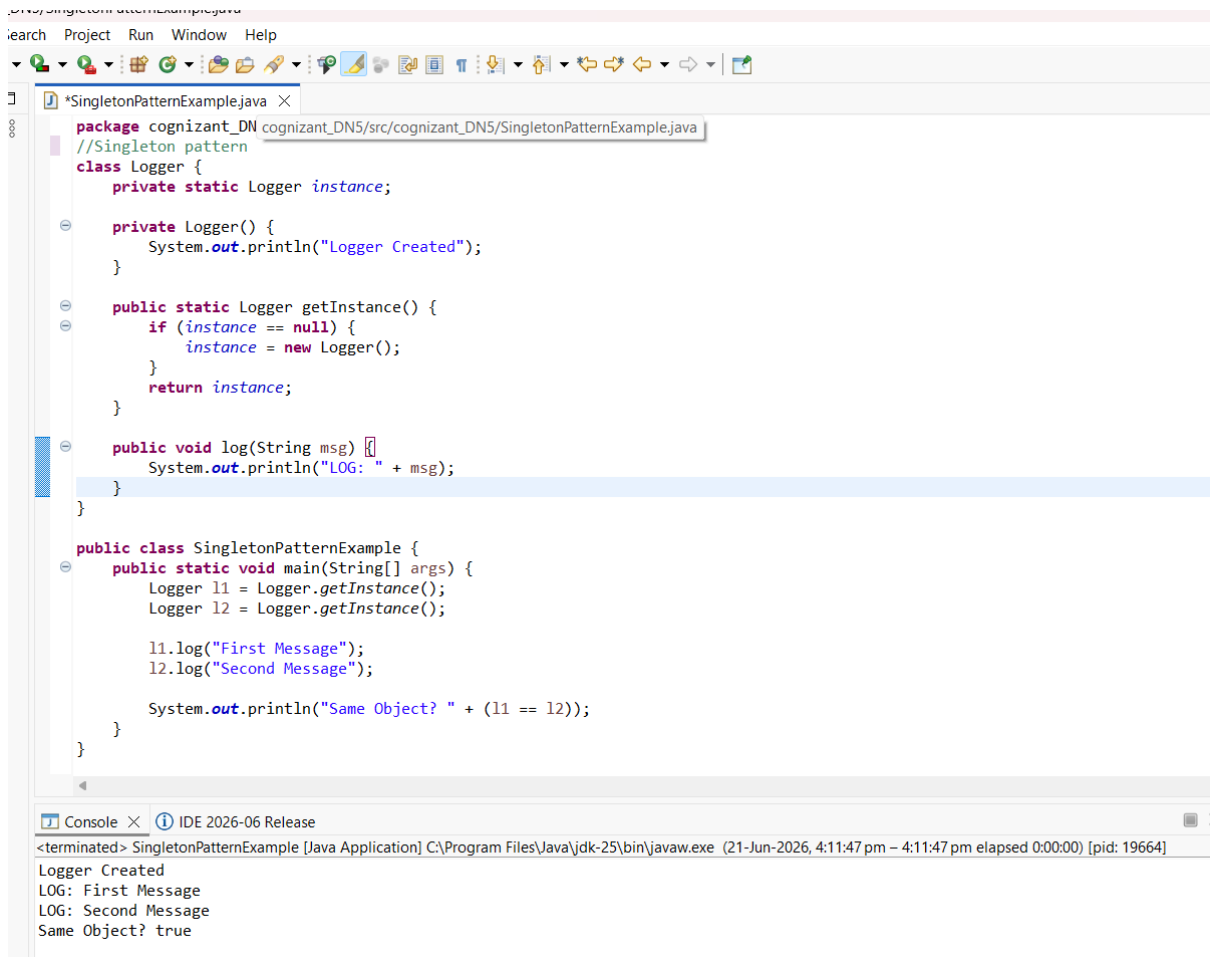


Week 1 : Design Patterns and Principles

Exercise 1 : Singleton pattern



The screenshot displays an IDE window with a Java file named `*SingletonPatternExample.java`. The code implements the Singleton pattern for a `Logger` class. The `Logger` class has a private static `instance` and a private constructor that prints "Logger Created". The `getInstance()` method checks if the `instance` is null and creates a new instance if necessary. The `log()` method prints a message. The `SingletonPatternExample` class has a `main` method that creates two references to the `Logger` instance and prints "Same Object? " followed by the result of `(l1 == l2)`.

```
package cognizant_DN cognizant_DN5/src/cognizant_DN5/SingletonPatternExample.java
//Singleton pattern
class Logger {
    private static Logger instance;

    private Logger() {
        System.out.println("Logger Created");
    }

    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    public void log(String msg) {
        System.out.println("LOG: " + msg);
    }
}

public class SingletonPatternExample {
    public static void main(String[] args) {
        Logger l1 = Logger.getInstance();
        Logger l2 = Logger.getInstance();

        l1.log("First Message");
        l2.log("Second Message");

        System.out.println("Same Object? " + (l1 == l2));
    }
}
```

The console output shows the following messages:

```
<terminated> SingletonPatternExample [Java Application] C:\Program Files\Java\jdk-25\bin\javaw.exe (21-Jun-2026, 4:11:47 pm - 4:11:47 pm elapsed 0:00:00) [pid: 19664]
Logger Created
LOG: First Message
LOG: Second Message
Same Object? true
```

Exercise 2 : Factory Method Pattern

```
SingletonPatternExample.java  FactoryMethodPatternExample.java ×
package cognizant_DN5;

interface Document {
    void open();
}

class WordDocument implements Document {
    public void open() {
        System.out.println("Opening Word Document");
    }
}

class PdfDocument implements Document {
    public void open() {
        System.out.println("Opening PDF Document");
    }
}

class ExcelDocument implements Document {
    public void open() {
        System.out.println("Opening Excel Document");
    }
}

abstract class DocumentFactory {
    abstract Document createDocument();
}

class WordFactory extends DocumentFactory {
    Document createDocument() {
        return new WordDocument();
    }
}

class PdfFactory extends DocumentFactory {
    Document createDocument() {
        return new PdfDocument();
    }
}

class ExcelFactory extends DocumentFactory {
    Document createDocument() {
        return new ExcelDocument();
    }
}

public class FactoryMethodPatternExample {
    public static void main(String[] args) {
        DocumentFactory factory;

        factory = new WordFactory();
        factory.createDocument().open();

        factory = new PdfFactory();
        factory.createDocument().open();

        factory = new ExcelFactory();
        factory.createDocument().open();
    }
}
```

Console × IDE 2026-06 Release

<terminated> FactoryMethodPatternExample [Java Application] C:\Program Files\Java\jdk-25\bin\javaw.exe

Opening Word Document
Opening PDF Document
Opening Excel Document

Exercise 3: Builder Pattern Example

```
SingletonPatternExample.java BuilderPatternExample.java X
package cognizant_DN5;

class Computer {
    private String cpu;
    private int ram;
    private int storage;

    private Computer(Builder builder) {
        cpu = builder.cpu;
        ram = builder.ram;
        storage = builder.storage;
    }

    void display() {
        System.out.println("CPU: " + cpu);
        System.out.println("RAM: " + ram + " GB");
        System.out.println("Storage: " + storage + " GB");
    }

    static class Builder {
        private String cpu;
        private int ram;
        private int storage;

        Builder setCpu(String cpu) {
            this.cpu = cpu;
            return this;
        }

        Builder setRam(int ram) {
            this.ram = ram;
            return this;
        }

        Builder setStorage(int storage) {
            this.storage = storage;
            return this;
        }

        Computer build() {
            return new Computer(this);
        }
    }
}

public class BuilderPatternExample {
    public static void main(String[] args) {
        Computer c = new Computer.Builder()
            .setCpu("Intel i7")
            .setRam(16)
            .setStorage(512)
            .build();

        c.display();
    }
}

Console X IDE 2026-06 Release
<terminated> BuilderPatternExample [Java Application] C:\Program Files\Java\jdk-25\bin\javaw.exe (21-Jul-2025 10:00:00 AM)
CPU: Intel i7
RAM: 16 GB
Storage: 512 GB
```

Exercise 4: Adapter Pattern Example



```
package cognizant_DN5;

interface PaymentProcessor {
    void processPayment(double amount);
}

class PayPalGateway {
    void makePayment(double amount) {
        System.out.println("Paid ₹" + amount + " using PayPal");
    }
}

class GooglePayGateway {
    void sendMoney(double amount) {
        System.out.println("Paid ₹" + amount + " using Google Pay");
    }
}

class PayPalAdapter implements PaymentProcessor {
    private PayPalGateway gateway;

    PayPalAdapter(PayPalGateway gateway) {
        this.gateway = gateway;
    }

    public void processPayment(double amount) {
        gateway.makePayment(amount);
    }
}

class GooglePayAdapter implements PaymentProcessor {
    private GooglePayGateway gateway;

    GooglePayAdapter(GooglePayGateway gateway) {
        this.gateway = gateway;
    }

    public void processPayment(double amount) {
        gateway.sendMoney(amount);
    }
}

public void processPayment(double amount) {
    gateway.sendMoney(amount);
}

public class AdapterPattern {
    public static void main(String[] args) {
        PaymentProcessor p1 =
            new PayPalAdapter(new PayPalGateway());

        PaymentProcessor p2 =
            new GooglePayAdapter(new GooglePayGateway());

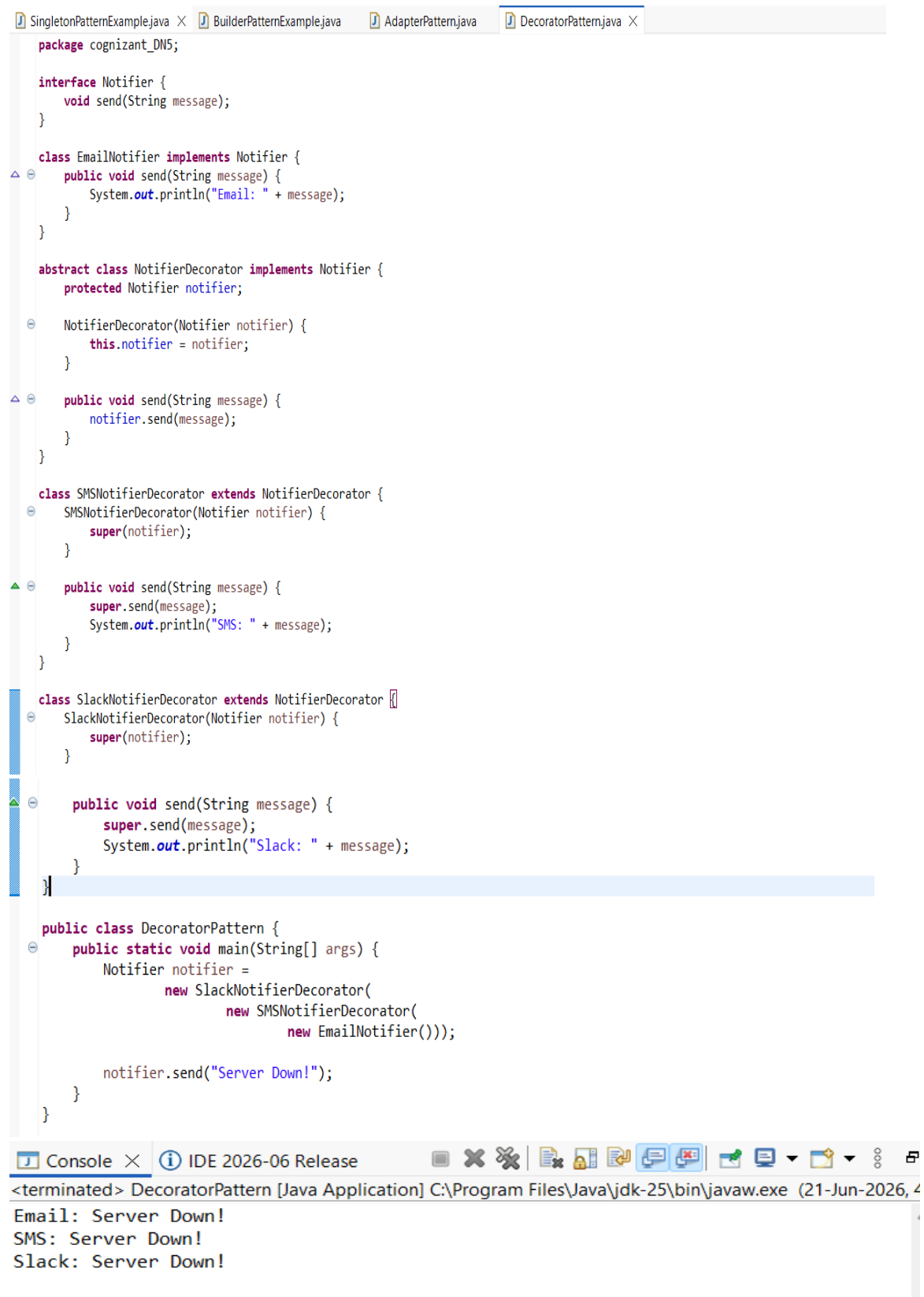
        p1.processPayment(500);
        p2.processPayment(1000);
    }
}
```

Console × IDE 2026-06 Release

<terminated> AdapterPattern [Java Application] C:\Program Files\Java\jdk-25\bin\javaw.exe (21-Jun-2026, 4:23:)

Paid ₹500.0 using PayPal
Paid ₹1000.0 using Google Pay

Exercise 5: Decorator Pattern Example



```
package cognizant_DMS;

interface Notifier {
    void send(String message);
}

class EmailNotifier implements Notifier {
    public void send(String message) {
        System.out.println("Email: " + message);
    }
}

abstract class NotifierDecorator implements Notifier {
    protected Notifier notifier;

    NotifierDecorator(Notifier notifier) {
        this.notifier = notifier;
    }

    public void send(String message) {
        notifier.send(message);
    }
}

class SMSNotifierDecorator extends NotifierDecorator {
    SMSNotifierDecorator(Notifier notifier) {
        super(notifier);
    }

    public void send(String message) {
        super.send(message);
        System.out.println("SMS: " + message);
    }
}

class SlackNotifierDecorator extends NotifierDecorator {
    SlackNotifierDecorator(Notifier notifier) {
        super(notifier);
    }

    public void send(String message) {
        super.send(message);
        System.out.println("Slack: " + message);
    }
}

public class DecoratorPattern {
    public static void main(String[] args) {
        Notifier notifier =
            new SlackNotifierDecorator(
                new SMSNotifierDecorator(
                    new EmailNotifier()));

        notifier.send("Server Down!");
    }
}
```

Console > IDE 2026-06 Release

<terminated> DecoratorPattern [Java Application] C:\Program Files\Java\jdk-25\bin\javaw.exe (21-Jun-2026, 4

Email: Server Down!
SMS: Server Down!
Slack: Server Down!

Exercise 6: Proxy Pattern Example

The screenshot displays an IDE window with the file `ProxyPattern.java` selected. The code implements the Proxy Pattern for an `Image` interface. It includes a `RealImage` class that loads and displays images, and a `ProxyImage` class that acts as a placeholder, delegating the load and display actions to a `RealImage` instance. The `ProxyPattern` class contains a `main` method that demonstrates the usage of the proxy.

```
package cognizant_DN5;

interface Image {
    void display();
}

class RealImage implements Image {
    private String fileName;

    RealImage(String fileName) {
        this.fileName = fileName;
        load();
    }

    private void load() {
        System.out.println("Loading " + fileName + " from server...");
    }

    public void display() {
        System.out.println("Displaying " + fileName);
    }
}

class ProxyImage implements Image {
    private RealImage realImage;
    private String fileName;

    ProxyImage(String fileName) {
        this.fileName = fileName;
    }

    public void display() {
        if (realImage == null) {
            realImage = new RealImage(fileName);
        }
        realImage.display();
    }
}

public class ProxyPattern {
    public static void main(String[] args) {
        Image image = new ProxyImage("nature.jpg");

        image.display();
        image.display();
    }
}
```

The console output shows the execution of the `main` method, resulting in the following sequence of messages:

```
<terminated> ProxyPattern [Java Application] C:\Program Files\Java\jdk-25\bin\javaw.exe (21-Jun-2026, 4:27:14)
Loading nature.jpg from server...
Displaying nature.jpg
Displaying nature.jpg
```

Exercise 7: Observer Pattern Example

```
SingletonPatternExample.java BuilderPatternExample.java AdapterPattern.java DecoratorPattern.java ProxyPattern.java ObserverPattern.java X
package cognizant_DNS;

import java.util.*;

interface Observer {
    void update(double price);
}

interface Stock {
    void registerObserver(Observer o);
    void removeObserver(Observer o);
    void notifyObservers();
}

class StockMarket implements Stock {
    private List<Observer> observers = new ArrayList<>();
    private double price;

    public void setPrice(double price) {
        this.price = price;
        notifyObservers();
    }

    public void registerObserver(Observer o) {
        observers.add(o);
    }

    public void removeObserver(Observer o) {
        observers.remove(o);
    }

    public void notifyObservers() {
        for (Observer o : observers) {
            o.update(price);
        }
    }
}

class MobileApp implements Observer {
    public void update(double price) {
        System.out.println("Mobile App: Price = ₹" + price);
    }
}

class WebApp implements Observer {
    public void update(double price) {
        System.out.println("Web App: Price = ₹" + price);
    }
}

public class ObserverPattern {
    public static void main(String[] args) {
        StockMarket market = new StockMarket();

        Observer mobile = new MobileApp();
        Observer web = new WebApp();

        market.registerObserver(mobile);
        market.registerObserver(web);

        market.setPrice(1500);
        market.setPrice(1700);
    }
}
```

Console X IDE 2026-06 Release

<terminated> ObserverPattern [Java Application] C:\Program Files\Java\jdk-25\bin\javaw.exe (21-Jun-2026, 4:..

```
Mobile App: Price = ₹1500.0
Web App: Price = ₹1500.0
Mobile App: Price = ₹1700.0
Web App: Price = ₹1700.0
```